

工厂设计模式实验指导书

一、实验目的

1. 掌握创建型设计模式的核心思想及应用场景
2. 深入理解并实现三种工厂模式（简单工厂模式、工厂方法模式、抽象工厂模式）
3. 学会在华为CodeArts IDE中进行Java项目开发与UML类图绘制
4. 理解工厂模式在实际框架（如Spring）中的应用

二、环境准备

2.1 软件环境

- 操作系统：Windows 10/11（64位）
- 开发工具：华为CodeArts IDE for Java（2.3.0及以上版本）
- JDK版本：JDK 8或11
- 构建工具：Maven 3.8.x
- UML绘图工具：CodeArts内置架构建模工具

2.2 环境配置步骤

2.2.1 安装华为CodeArts IDE

1. 访问华为云官网下载CodeArts IDE for Java安装包
2. 运行安装程序，按照向导完成安装：
 - 选择安装路径（建议默认路径）
 - 勾选"创建桌面快捷方式"
 - 等待安装完成并启动IDE

2.2.2 登录与初始化

1. 首次启动IDE后，点击右上角"登录"按钮
2. 使用华为云账号登录（需提前完成实名认证）
3. 等待Java语言服务初始化（右下角状态栏显示"Java Language Server is ready"）

2.2.3 创建Java项目

1. 通过三种方式创建项目：
 - 菜单方式：文件 -> 新建 -> 工程
 - 欢迎界面：点击"新建项目"或"创建Java工程"
 - 快捷键：Alt+P
2. 在新建工程向导中：
 - 选择构建系统：Maven
 - 选择框架：None
 - 输入项目名称：ElectronicProductFactory
 - 选择项目存储路径
3. 点击"确定"完成创建

三、实验内容

3.1 简单工厂模式实现

3.1.1 实验场景

实现一个基础电子产品工厂，能够根据客户端请求生产不同类型的电子产品（手机/平板/笔记本）

3.1.2 UML类图绘制

1. 在CodeArts IDE中打开架构建模工具：
 - 菜单：视图 -> 架构建模
 - 在建模面板中选择"UML类图"
2. 绘制简单工厂模式的UML类图：
 - 抽象产品：ElectronicProduct
 - 具体产品：MobilePhone, Tablet, Laptop
 - 工厂类：ProductFactory

3.1.3 代码实现

步骤1：创建抽象产品接口

java

```
// ElectronicProduct.java
public interface ElectronicProduct {
    void produce();
    String getProductInfo();
}
```

步骤2: 实现具体产品类

```
// MobilePhone.java
public class MobilePhone implements ElectronicProduct {
    @Override
    public void produce() {
        System.out.println("生产智能手机");
    }

    @Override
    public String getProductInfo() {
        return "智能手机 - 通信与娱乐设备";
    }
}
```

```
// Tablet.java
public class Tablet implements ElectronicProduct {
    @Override
    public void produce() {
        System.out.println("生产平板电脑");
    }

    @Override
    public String getProductInfo() {
        return "平板电脑 - 便携办公娱乐设备";
    }
}
```

```
// Laptop.java
public class Laptop implements ElectronicProduct {
    @Override
    public void produce() {
        System.out.println("生产笔记本电脑");
    }

    @Override
    public String getProductInfo() {
```

```
java    return "笔记本电脑 - 移动办公设备";
        }
    }
```

步骤3: 创建简单工厂类

```
java
// ProductFactory.java
public class ProductFactory {
    public ElectronicProduct createProduct(String type) {
        if (type == null || type.isEmpty()) {
            return null;
        }

        switch (type.toLowerCase()) {
            case "mobilephone":
                return new MobilePhone();
            case "tablet":
                return new Tablet();
            case "laptop":
                return new Laptop();
            default:
                throw new IllegalArgumentException("不支持的产品类型:" + type);
        }
    }
}
```

步骤4: 编写客户端测试代码

```
// Client.java
public class Client {
    public static void main(String[] args) {
        ProductFactory factory = new ProductFactory();

        ElectronicProduct phone = factory.createProduct("mobilephone");
        phone.produce();
        System.out.println(phone.getProductInfo());

        ElectronicProduct tablet = factory.createProduct("tablet");
        tablet.produce();
        System.out.println(tablet.getProductInfo());

        ElectronicProduct laptop = factory.createProduct("laptop");
```

```
java        laptop.produce();
            System.out.println(laptop.getProductInfo());
        }
    }
```

3.1.4 实验任务分析

1. 添加新产品类型的修改分析：

- 需要修改ProductFactory类的createProduct方法，添加新的case分支
- 需要创建新的具体产品类实现ElectronicProduct接口
- 违反开闭原则，需要修改工厂类代码

2. 简单工厂模式优缺点：

- 优点：客户端无需知道产品创建细节，只需提供产品类型；集中管理产品创建逻辑
- 缺点：新增产品需要修改工厂类，违反开闭原则；工厂类职责过重，复杂度高

3.2 工厂方法模式实现

3.2.1 实验场景

扩展系统以支持不同品牌的电子产品生产，每个品牌有自己的工厂

3.2.2 UML类图绘制

1. 在架构建模工具中创建新的UML类图

2. 绘制工厂方法模式的UML类图：

- 抽象产品：ElectronicProduct
- 具体产品：ApplePhone, AppleTablet, SamsungPhone, SamsungTablet
- 抽象工厂：BrandFactory
- 具体工厂：AppleFactory, SamsungFactory

3.2.3 代码实现

步骤1：创建抽象产品接口（复用3.1.3中的ElectronicProduct）

步骤2：创建品牌产品类

// ApplePhone.java

```
public class ApplePhone implements ElectronicProduct {  
    @Override  
    public void produce() {  
        System.out.println("生产Apple手机");  
    }  
  
    @Override  
    public String getProductInfo() {  
        return "Apple iPhone – iOS系统智能手机";  
    }  
}
```

// AppleTablet.java

```
public class AppleTablet implements ElectronicProduct {  
    @Override  
    public void produce() {  
        System.out.println("生产Apple平板电脑");  
    }  
  
    @Override  
    public String getProductInfo() {  
        return "Apple iPad – iOS系统平板电脑";  
    }  
}
```

// SamsungPhone.java

```
public class SamsungPhone implements ElectronicProduct {  
    @Override  
    public void produce() {  
        System.out.println("生产Samsung手机");  
    }  
  
    @Override  
    public String getProductInfo() {  
        return "Samsung Galaxy – Android系统智能手机";  
    }  
}
```

// SamsungTablet.java

```
public class SamsungTablet implements ElectronicProduct {  
    @Override  
    public void produce() {  
        System.out.println("生产Samsung平板电脑");  
    }  
}
```

```
java
    @Override
    public String getProductInfo() {
        return "Samsung Galaxy Tab – Android系统平板电脑";
    }
}
```

步骤3：创建抽象工厂接口

```
java
// BrandFactory.java
public interface BrandFactory {
    ElectronicProduct createPhone();
    ElectronicProduct createTablet();
}
```

步骤4：实现具体工厂类

java

```
// AppleFactory.java
public class AppleFactory implements BrandFactory {
    @Override
    public ElectronicProduct createPhone() {
        return new ApplePhone();
    }

    @Override
    public ElectronicProduct createTablet() {
        return new AppleTablet();
    }
}

// SamsungFactory.java
public class SamsungFactory implements BrandFactory {
    @Override
    public ElectronicProduct createPhone() {
        return new SamsungPhone();
    }

    @Override
    public ElectronicProduct createTablet() {
        return new SamsungTablet();
    }
}
```

步骤5: 客户端测试代码

```
// FactoryMethodClient.java
public class FactoryMethodClient {
    public static void main(String[] args) {
        BrandFactory appleFactory = new AppleFactory();
        ElectronicProduct iphone = appleFactory.createPhone();
        iphone.produce();
        System.out.println(iphone.getProductInfo());

        ElectronicProduct ipad = appleFactory.createTablet();
        ipad.produce();
        System.out.println(ipad.getProductInfo());

        BrandFactory samsungFactory = new SamsungFactory();
        ElectronicProduct galaxyPhone = samsungFactory.createPhone();
        galaxyPhone.produce();
    }
}
```



```

java      System.out.println(galaxyPhone.getProductInfo());

          ElectronicProduct galaxyTab = samsungFactory.createTablet();
          galaxyTab.produce();
          System.out.println(galaxyTab.getProductInfo());
      }
  }

```

3.2.4 实验任务分析

1. 添加新品牌（Huawei）的扩展步骤：

- 创建HuaweiPhone和HuaweiTablet类实现ElectronicProduct接口
- 创建HuaweiFactory类实现BrandFactory接口
- 实现createPhone()和createTablet()方法
- 客户端直接使用新的工厂类，无需修改现有代码

2. 工厂方法模式与简单工厂模式的区别：

- 工厂方法模式将具体产品创建延迟到子类，遵循开闭原则
- 简单工厂模式集中在一个工厂类创建所有产品，扩展性差
- 工厂方法模式每个产品由专门的工厂创建，职责单一

3. 开闭原则的体现：

- 新增产品品牌时，只需添加新的具体产品和具体工厂类
- 无需修改现有工厂和产品代码
- 客户端可以使用新的工厂类，原有代码保持不变

3.3 抽象工厂模式实现

3.3.1 实验场景

进一步扩展系统，每个品牌工厂不仅能生产手机和平板，还能生产配套的配件（如充电器、耳机）

3.3.2 UML类图绘制

1. 在架构建模工具中创建新的UML类图

2. 绘制抽象工厂模式的UML类图：

- 抽象产品族：Phone, Tablet, Charger, Headphone
- 具体产品：各品牌对应的产品实现
- 抽象工厂：ElectronicProductFactory

- 具体工厂：AppleFactory, SamsungFactory

3.3.3 代码实现

步骤1：创建产品族接口

```
java
// Phone.java
public interface Phone {
    void produce();
    String getPhoneInfo();
}

// Tablet.java
public interface Tablet {
    void produce();
    String getTabletInfo();
}

// Charger.java
public interface Charger {
    void produce();
    String getChargerInfo();
}

// Headphone.java
public interface Headphone {
    void produce();
    String getHeadphoneInfo();
}
```

步骤2：实现品牌产品族

```
// Apple产品族实现
public class ApplePhone implements Phone {
    @Override
    public void produce() {
        System.out.println("生产Apple手机");
    }

    @Override
    public String getPhoneInfo() {
        return "Apple iPhone";
    }
}
```

```

java    }
    }

    public class AppleTablet implements Tablet {
        @Override
        public void produce() {
            System.out.println("生产Apple平板电脑");
        }

        @Override
        public String getTabletInfo() {
            return "Apple iPad";
        }
    }

    public class AppleCharger implements Charger {
        @Override
        public void produce() {
            System.out.println("生产Apple充电器");
        }

        @Override
        public String getChargerInfo() {
            return "Apple 20W USB-C充电器";
        }
    }

    public class AppleHeadphone implements Headphone {
        @Override
        public void produce() {
            System.out.println("生产Apple耳机");
        }

        @Override
        public String getHeadphoneInfo() {
            return "Apple AirPods Pro";
        }
    }

    // Samsung产品族实现（代码省略，类似Apple产品族）

```

步骤3：创建抽象工厂接口

java

```
// ElectronicProductFactory.java
public interface ElectronicProductFactory {
    Phone createPhone();
    Tablet createTablet();
    Charger createCharger();
    Headphone createHeadphone();
}
```

步骤4: 实现具体工厂

java

```
// AppleFactory.java
public class AppleFactory implements ElectronicProductFactory {
    @Override
    public Phone createPhone() {
        return new ApplePhone();
    }

    @Override
    public Tablet createTablet() {
        return new AppleTablet();
    }

    @Override
    public Charger createCharger() {
        return new AppleCharger();
    }

    @Override
    public Headphone createHeadphone() {
        return new AppleHeadphone();
    }
}
```

// SamsungFactory.java (代码省略, 类似AppleFactory)

步骤5: 客户端测试代码

```
// AbstractFactoryClient.java
public class AbstractFactoryClient {
    public static void main(String[] args) {
```

```

java    ElectronicProductFactory appleFactory = new AppleFactory();
        System.out.println("=== Apple产品族 ===");
        appleFactory.createPhone().produce();
        appleFactory.createTablet().produce();
        appleFactory.createCharger().produce();
        appleFactory.createHeadphone().produce();

        ElectronicProductFactory samsungFactory = new SamsungFactory();
        System.out.println("\n=== Samsung产品族 ===");
        samsungFactory.createPhone().produce();
        samsungFactory.createTablet().produce();
        samsungFactory.createCharger().produce();
        samsungFactory.createHeadphone().produce();
    }
}

```

3.3.4 实验任务分析

1. 产品族概念：

- 产品族是指位于不同产品等级结构中，功能相关联的产品组成的家族
- 在抽象工厂模式中，每个具体工厂生产一整套产品族
- 如AppleFactory生产Apple的手机、平板、充电器和耳机组成的产品族

2. 添加新产品族（智能手表）的修改：

- 添加Watch接口（抽象产品）
- 在ElectronicProductFactory接口中添加createWatch()方法
- 所有具体工厂类都需要实现createWatch()方法
- 违反开闭原则，这是抽象工厂模式的局限性

3. 抽象工厂模式与工厂方法模式的区别：

- 抽象工厂模式生产产品族，工厂方法模式生产单一产品
- 抽象工厂模式中具体工厂生产多个相关产品，工厂方法模式中具体工厂只生产一种产品
- 抽象工厂模式扩展产品族困难，扩展新产品等级结构相对容易
- 工厂方法模式扩展新产品容易，扩展产品族困难

四、华为云平台配置（选做）

4.1 代码托管配置

1. 在CodeArts IDE中点击"代码 -> 代码托管"

2. 点击"新建仓库", 选择"按模板新建"
3. 输入仓库名称"electronic-factory", 点击"确定"
4. 将本地代码提交到远程仓库:

```
bash
git add .
git commit -m "完成三种工厂模式实现"
git push origin main
```

4.2 代码检查

1. 点击"代码 -> 代码检查"
2. 选择当前项目, 点击"立即执行"
3. 查看检查结果, 修复提示的代码问题
4. 根据检查建议优化代码结构

五、实验报告要求

5.1 实验内容要求

1. 源代码实现:
 - 三种工厂模式的完整代码
 - 客户端测试代码
2. UML类图:
 - 简单工厂模式类图
 - 工厂方法模式类图
 - 抽象工厂模式类图
3. 功能验证:
 - 运行测试代码的输出结果截图

5.2 分析讨论要求

1. 三种工厂模式的优缺点比较
2. 不同工厂模式的适用场景分析
3. 工厂模式如何实现创建逻辑与使用逻辑的分离

4. 工厂模式对系统可扩展性的影响

5.3 扩展思考要求

1. 添加新品牌（如小米）时，哪种工厂模式最容易扩展？为什么？
2. 添加新产品类型（如智能手表）时，哪种工厂模式最容易扩展？为什么？
3. 讨论工厂模式在Spring框架中的应用（BeanFactory）

三、实验内容（续）

3.3 抽象工厂模式实现（续）

3.3.3 代码实现（续）

步骤2：实现品牌产品族（续）

```
// SamsungPhone.java
public class SamsungPhone implements Phone {
    @Override
    public void produce() {
        System.out.println("生产Samsung手机");
    }

    @Override
    public String getPhoneInfo() {
        return "Samsung Galaxy S23";
    }
}

// SamsungTablet.java
public class SamsungTablet implements Tablet {
    @Override
    public void produce() {
        System.out.println("生产Samsung平板电脑");
    }

    @Override
    public String getTabletInfo() {
        return "Samsung Galaxy Tab S9";
    }
}
```

```

java// SamsungCharger.java
public class SamsungCharger implements Charger {
    @Override
    public void produce() {
        System.out.println("生产Samsung充电器");
    }

    @Override
    public String getChargerInfo() {
        return "Samsung 25W Super Fast Charger";
    }
}

// SamsungHeadphone.java
public class SamsungHeadphone implements Headphone {
    @Override
    public void produce() {
        System.out.println("生产Samsung耳机");
    }

    @Override
    public String getHeadphoneInfo() {
        return "Samsung Galaxy Buds Pro";
    }
}

```

步骤4: 实现具体工厂 (续)

```

// SamsungFactory.java
public class SamsungFactory implements ElectronicProductFactory {
    @Override
    public Phone createPhone() {
        return new SamsungPhone();
    }

    @Override
    public Tablet createTablet() {
        return new SamsungTablet();
    }

    @Override
    public Charger createCharger() {
        return new SamsungCharger();
    }
}

```



```
java
    @Override
    public Headphone createHeadphone() {
        return new SamsungHeadphone();
    }
}
```

四、华为云平台配置（续）

4.1 代码托管配置（续）

5. 配置.gitignore文件，排除不需要提交的文件：

```
# Maven
target/
pom.xml.tag
pom.xml.releaseBackup
pom.xml.versionsBackup
pom.xml.next
release.properties
dependency-reduced-pom.xml
buildNumber.properties
.mvn/timing.properties

# IDE
.idea/
.jlsp/
*.iml
*.iws
*.ipr
.DS_Store
```

4.2 编译构建配置

1. 点击"持续交付 -> 编译构建"
2. 点击"新建任务"，配置任务信息：
 - 任务名称：electronic-factory-build
 - 源码源：选择代码托管中的仓库
 - 分支：main

3. 选择构建模板：Maven

4. 配置构建命令：

```
bash
mvn clean package -DskipTests
```

5. 点击"新建并执行"，查看构建结果

4.3 部署配置

1. 点击"持续交付 -> 部署"

2. 点击"新建应用"，输入应用名称"electronic-factory-app"

3. 选择"空白模板"，点击"下一步"

4. 配置部署步骤：

- 选择部署来源：编译构建任务
- 选择构建任务：electronic-factory-build
- 部署脚本：

```
bash
#!/bin/bash
java -jar target/electronic-factory-1.0.0.jar
```

5. 点击"保存并部署"，查看部署结果

五、实验报告要求（续）

5.2 分析讨论要求（续）

5. 工厂模式在Spring框架中的应用分析：

- Spring IoC容器（ BeanFactory/Application Context ）使用了工厂模式
- BeanFactory作为抽象工厂，负责创建和管理Bean实例
- 具体工厂实现（ 如DefaultListableBeanFactory ）负责实例化Bean
- FactoryBean接口允许自定义Bean的创建逻辑

5.3 扩展思考要求（续）

4. 工厂模式在实际项目中的应用案例分析：

- 日志框架中的LoggerFactory
- 数据库连接池的创建
- 解析不同格式文件（XML/JSON）的解析器工厂

三、实验内容（补充）

UML类图绘制详细步骤

使用CodeArts IDE架构建模工具绘制UML类图

1. 打开架构建模工具

- 在CodeArts IDE顶部菜单中选择 **视图 > 架构建模**
- 或使用快捷键 **Alt+Shift+M** 快速打开
- 首次使用需同意用户协议并启用相关插件

2. 创建UML类图文件

- 在左侧项目资源管理器中右键点击 **src/main/java** 目录
- 选择 **新建 > 架构建模 > UML类图**
- 输入文件名（如 **SimpleFactoryUML**）并点击 **确定**

3. 绘制类图元素

- 从右侧工具栏拖拽 **Class** 元素到画布创建类
- 双击类元素编辑名称、属性和方法
- 使用 **Association** 工具创建类之间的关联关系
- 使用 **Generalization** 工具创建继承关系

4. 从类图生成Java代码

- 右键点击画布空白处，选择 **生成 > Java代码**
- 在弹出对话框中选择生成路径（建议默认 **src/main/java**）
- 勾选需要生成的类，点击 **确定**
- 系统自动生成包含包结构的Java代码文件

5. 导出UML类图

- 右键点击画布空白处，选择 **导出 > 图片**

- 选择保存格式（PNG/JPG/SVG）和路径
- 调整图片分辨率和大小，点击 确定

三种工厂模式对比分析

特性	简单工厂模式	工厂方法模式	抽象工厂模式
核心思想	单一工厂生产所有产品	每个产品由专属工厂生产	生产完整产品族
产品扩展性	差（需修改工厂类）	好（新增产品+工厂）	产品族扩展困难
工厂职责	过重（生产所有产品）	单一（生产一种产品）	中等（生产一族产品）
开闭原则	不遵循	遵循	部分遵循
适用场景	产品类型少且稳定	产品类型多变	需统一产品族
实现复杂度	低	中	高

Spring框架中工厂模式的应用

Spring IoC容器核心采用**抽象工厂模式**，主要实现类关系如下：

1. 抽象工厂： `BeanFactory` 接口

```
java
public interface BeanFactory {
    Object getBean(String name) throws BeansException;
    <T> T getBean(Class<T> requiredType) throws BeansException;
    // 其他方法...
}
```

2. 具体工厂： `DefaultListableBeanFactory`

```
java
public class DefaultListableBeanFactory extends AbstractAutowireCapableBeanFactory
    implements ConfigurableListableBeanFactory, BeanDefinitionRegistry {
    // 实现Bean创建和管理逻辑
}
```

3. 产品：所有Spring管理的Bean

```
java
@Service
public class UserService {
    // 业务逻辑...
}
```

4. FactoryBean扩展点：允许自定义Bean创建逻辑

```
java
public interface FactoryBean<T> {
    T getObject() throws Exception;
    Class<?> getObjectType();
    boolean isSingleton();
}
```

六、常见问题解决

6.1 CodeArts IDE相关

1. Java语言服务初始化失败

- 解决方案：检查JDK配置，菜单 文件 > 设置 > Java > 已安装的JDK
- 确保JDK路径正确，建议使用JDK 8或11版本

2. UML类图无法生成代码

- 解决方案：确认类图元素包含必要信息（类名、包名）
- 检查项目构建路径，确保生成目录在源码根目录下

6.2 代码编译错误

1. 包不存在错误

- 解决方案：右键项目 Maven > 重新加载项目
- 检查pom.xml依赖配置是否正确

2. 抽象方法未实现

- 解决方案：确保所有具体工厂类实现了抽象工厂的所有方法
- 使用IDE自动实现功能（快捷键 **Alt+Enter**）

6.3 华为云平台相关

1. 代码提交失败

- 解决方案：检查网络连接，确认华为云账号已登录
- 执行命令 `git remote -v` 验证远程仓库地址

2. 构建任务执行失败

- 解决方案：查看构建日志，检查Maven依赖是否完整
- 本地执行 `mvn clean package` 验证构建过程

七、实验总结

7.1 实验核心价值

通过本次实验，深入理解了工厂模式的三种实现方式及其在软件开发中的重要作用：

1. **解耦创建与使用**：工厂模式将对象创建逻辑与使用逻辑分离，客户端无需关注对象创建细节，只需通过工厂接口获取产品实例。
2. **提升可维护性**：集中管理对象创建逻辑，当产品实现变化时，只需修改对应工厂或产品代码，无需修改客户端。
3. **增强可扩展性**：通过抽象工厂和具体工厂的分层设计，系统可以在不修改现有代码的情况下引入新的产品类型或产品族。
4. **框架设计应用**：理解工厂模式在Spring等主流框架中的应用，为学习复杂框架源码奠定基础。

7.2 关键知识点回顾

- **简单工厂模式**：适用于产品类型较少且变化不频繁的场景，如工具类库设计。
- **工厂方法模式**：适用于产品类型多变的场景，如插件化架构设计。
- **抽象工厂模式**：适用于需要协调多个相关产品的场景，如跨平台组件开发。
- **开闭原则**：工厂方法模式和抽象工厂模式通过扩展而非修改实现开闭原则。

7.3 实际开发建议

1. **合理选择工厂模式**：根据产品复杂度和变化频率选择合适的工厂模式，避免过度设计。
2. **结合依赖注入**：在实际项目中，工厂模式常与依赖注入(DI)结合使用，如Spring框架。
3. **使用设计模式检测工具**：利用IDEA/CodeArts的设计模式检测插件，优化设计模式使用。
4. **文档化设计决策**：在代码注释和设计文档中说明使用工厂模式的原因和优势。

八、附录

8.1 完整代码目录结构

```
src/
├── main/
│   └── java/
│       └── com/
│           └── huawei/
│               ├── simplefactory/
│               │   ├── ElectronicProduct.java
│               │   ├── MobilePhone.java
│               │   ├── Tablet.java
│               │   ├── Laptop.java
│               │   ├── ProductFactory.java
│               │   └── Client.java
│               ├── factorymethod/
│               │   ├── ElectronicProduct.java
│               │   ├── ApplePhone.java
│               │   ├── AppleTablet.java
│               │   ├── SamsungPhone.java
│               │   ├── SamsungTablet.java
│               │   ├── BrandFactory.java
│               │   ├── AppleFactory.java
│               │   ├── SamsungFactory.java
│               │   └── FactoryMethodClient.java
│               └── abstractfactory/
│                   ├── product/
│                   │   ├── Phone.java
│                   │   ├── Tablet.java
│                   │   ├── Charger.java
│                   │   └── Headphone.java
│                   ├── apple/
│                   │   ├── ApplePhone.java
│                   │   ├── AppleTablet.java
│                   │   ├── AppleCharger.java
│                   │   └── AppleHeadphone.java
│                   ├── samsung/
│                   │   ├── SamsungPhone.java
│                   │   ├── SamsungTablet.java
│                   │   ├── SamsungCharger.java
│                   │   └── SamsungHeadphone.java
│                   ├── ElectronicProductFactory.java
│                   ├── AppleFactory.java
│                   └── SamsungFactory.java
```

```
└─ test/
  └─ java/
    └─ com/
      └─ huawei/
        └─ AbstractFactoryClient.java
        └─ SimpleFactoryTest.java
        └─ FactoryMethodTest.java
        └─ AbstractFactoryTest.java
```

8.2 华为CodeArts常用快捷键

功能	快捷键
新建Java项目	Alt+P
代码补全	Ctrl+空格
快速修复	Alt+Enter
运行程序	Ctrl+Shift+F10
调试程序	Shift+F9
生成UML类图	Alt+U
格式化代码	Ctrl+Alt+L
全局搜索	Ctrl+Shift+F